creationDate	modifiedDate	tags	p
2026-10-02 07:05	2026-10-02 07:05	architecture, book-notes, programming	[[Boc

Fundamentals of Software Architecture, 2nd Edition

What architecture is

- Software architecture is the structure of a system combined with the architecture characteristics it must support (the "-ilities"), the architecture decisions, and the design principles.
- "All architectures become iterative because of unknown unknowns. Agile just recognizes this and does it sooner." (Mark Richards)
- **First law of software architecture:** Everything in software architecture is a trade-off.
  - Corollary: if an architect thinks they've found a choice without trade-offs, they most likely just haven't identified the trade-off yet.
- **Second law of software architecture:** Why is more important than how.
- "Programmers know the benefits of everything, and the tradeoffs of nothing. Architects need to understand both." (Rich Hickey)
- "There are no wrong answers in architecture, only expensive ones." (Mark Richards)
- There's no best design in architecture, only a least-worst collection of trade-offs.
- It can be valuable to work out which architecture detail in a plan is the least important.

The architect's role

- Architects should still write production code, but avoid becoming a bottleneck by leaving key infrastructure and critical-path code to the development team.
- Writing production-grade prototype code is worthwhile too, both to keep good programming habits and because the prototype is likely to become a model for the team.
- Take part in frequent code reviews.
- Neal Ford's Architectural Katas site has small-group exercises for practicing architecture design.

- Document architecture decisions in Architecture Decision Records (ADRs). Nat Pryce's adr-tools is one way to manage them.
- Focus on the four C's of architecture: communication, collaboration, clarity, and conciseness.

Architecture styles

- The book rates each architecture style from one to five stars on a range of characteristics.
- The traditional layered architecture rates high (5 stars) on simplicity, cost, and ease of understanding, but low on most other characteristics.
- Service-based architecture is much coarser-grained than microservices. It's well rated in many areas, it's one of the most pragmatic choices, and it preserves ACID transactions better than finer-grained microservices.
- Asynchronous event handling increases throughput dramatically, but it makes error handling harder.
- If you feel the need to implement cross-service transactions in a microservices architecture, don't. It usually means your services are too fine-grained. Microservices are micro compared to a monolith, but they don't need to be tiny.
- Architectural leaps are often built in response to the pain points of previous architectures.
    - Software reuse sounds like a good idea, but service-oriented architecture (SOA), which was built to support it, taught a lot of hard lessons.
    - Microservices, where teams may write different services in different languages so reuse becomes impossible, are in some ways an SOA backlash. They have plenty of strengths of their own, though. They're inspired by domain-driven design and the bounded context, they rate very high on elasticity and scalability, and they're still very testable, unlike some other styles.